I checked your uploaded Python file, examination result.py. I'll prepare the complete activity record based on your actual code, in simple language suitable for a B.Tech 1st-year CSE Data Science student.

Your program accepts marks for five subjects, validates the marks, uses up to 5 grace marks to help subjects with marks between 30 and 34 reach 35, and calculates the total, average, and final result.

Examination Result Processing System

1. Problem statement

To develop a Python program that accepts marks for five subjects and checks whether the marks are valid. The program should provide up to 5 grace marks to eligible subjects, calculate the final marks, total, and average, and display whether the student has passed or failed.

2. Objectives

1. To accept marks for five subjects.
2. To validate marks within the range of 0 to 100.
3. To apply grace marks according to the given conditions.
4. To calculate the total and average marks.
5. To determine the final result as PASS or FAIL.
6. To understand lists, loops, conditional statements, and built-in functions in Python.

3. Assumptions

- The student has exactly five subjects.
- Each subject is marked out of 100.
- Marks are entered as integers.
- Marks must be between 0 and 100.
- A maximum of 5 grace marks is available for all subjects combined.
- Grace marks are applied only when a subject has marks from 30 to 34.
- A subject needs at least 35 marks to pass.
- The student passes only if all five final subject marks are at least 35.

4. Algorithm

1. Start the program.
2. Create an empty list called marks.
3. Input marks for five subjects and store them in the list.
4. Check whether every mark is between 0 and 100.
5. If any mark is invalid, display an error message.
6. Otherwise, set the available grace marks to 5.

7. Copy the original marks into a new list called final_marks.
8. Check each subject. If its marks are between 30 and 34, calculate the marks needed to reach 35.
9. Apply the required grace marks if enough grace marks remain.
10. Calculate the total and average of the final marks.
11. If every subject has at least 35 marks, display PASS; otherwise, display FAIL.
12. Stop the program.

5. Flowchart

Start

Input marks for 5 subjects

Are all marks between 0 and 100?

No

Display invalid marks message

Yes

Set grace marks = 5

Apply grace marks to eligible subjects

Calculate total and average

Are all final marks at least 35?

Display PASS or FAIL

End

Note: This flowchart is simplified. The grace-mark step processes each subject in order, and the final result is based on all five final marks.

6. Pseudocode

```
BEGIN
   marks = empty list

   FOR i = 1 TO 5
      INPUT mark
      ADD mark TO marks
   END FOR

   valid = TRUE
```

```
FOR each mark IN marks
    IF mark < 0 OR mark > 100
        valid = FALSE
    END IF
END FOR

IF valid = FALSE
    DISPLAY "Invalid marks"
ELSE
    grace = 5
    final_marks = copy of marks

    FOR each subject
        IF mark >= 30 AND mark < 35
            needed = 35 - mark

            IF needed <= grace
                mark = mark + needed
                grace = grace - needed
            END IF
        END IF
    END FOR

    total = SUM(final_marks)
    average = total / 5

    DISPLAY final_marks
    DISPLAY total
    DISPLAY average

    IF all final marks >= 35
        DISPLAY "PASS"
```

```
        ELSE
            DISPLAY "FAIL"
        END IF
    END IF
END
```

7. Python source code

```
marks = []

for i in range(5):
    mark = int(input("Enter mark for subject " + str(i + 1) + ": "))
    marks.append(mark)

valid = True
for mark in marks:
    if mark < 0 or mark > 100:
        valid = False

if not valid:
    print("Invalid marks: each mark must be 0 to 100.")
else:
    grace = 5
    final_marks = marks.copy()

    for i in range(5):
        if 30 <= final_marks[i] < 35:
            needed = 35 - final_marks[i]
            if needed <= grace:
                final_marks[i] += needed
                grace -= needed

    total = sum(final_marks)
```

```
    average = total / 5


    print("Final marks:", final_marks)
    print("Total:", total, "/ 500")
    print("Average:", round(average, 2), "%")


    if all(mark >= 35 for mark in final_marks):
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

8. Test cases

Test case	Input marks	Expected result
1	40, 50, 60, 70, 80	PASS; average 60%
2	32, 40, 50, 60, 70	PASS after 3 grace marks
3	30, 32, 40, 50, 60	FAIL; insufficient grace marks
4	40, 105, 50, 60, 70	Invalid marks message
5	35, 35, 35, 35, 35	PASS; average 35%

9. Sample output screenshots

These are sample console output previews, not actual screenshots captured from a Python interpreter.

Test 1 — Student passes without grace marks

```
Enter mark for subject 1: 40
Enter mark for subject 2: 50
Enter mark for subject 3: 60
Enter mark for subject 4: 70
Enter mark for subject 5: 80
Final marks: [40, 50, 60, 70, 80]
Total: 300 / 500
Average: 60.0 %
Result: PASS
```

Copy sample output

Test 2 — Student passes using grace marks

```
Enter mark for subject 1: 32
Enter mark for subject 2: 40
Enter mark for subject 3: 50
Enter mark for subject 4: 60
```

Enter mark for subject 5: 70
Final marks: [35, 40, 50, 60, 70]
Total: 255 / 500
Average: 51.0 %
Result: PASS

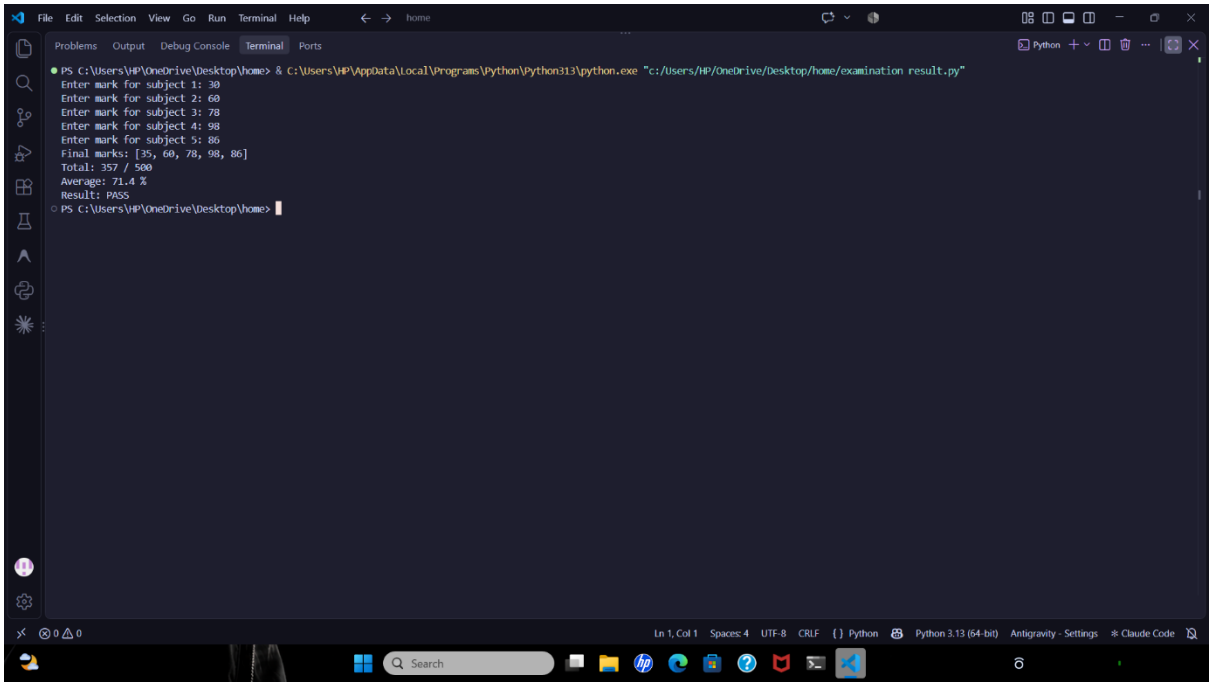
Copy sample output

Test 3 — Student fails

Enter mark for subject 1: 30
Enter mark for subject 2: 32
Enter mark for subject 3: 40
Enter mark for subject 4: 50
Enter mark for subject 5: 60
Final marks: [35, 32, 40, 50, 60]
Total: 217 / 500
Average: 43.4 %
Result: FAIL

Copy sample output

To get actual screenshots for your record, run the program in Python IDLE or VS Code and capture the output window.



10. Conclusion

The Examination Result Processing System was successfully developed using Python. The program accepts and validates marks for five subjects, applies available grace marks to eligible subjects, and calculates the final total and average. It determines whether the student has passed or failed based on the final marks. This program helps in understanding lists, loops, conditional statements, list copying, and built-in functions such as sum() and all() in Python.